Technical Specifications

for

PhytoPi

Version 1.0 approved

Prepared by Nolan Tuttle, Daniel Grijalva

PhytoPi

11/2/25

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

Table of Contents ii
Revision History ii
1. Introduction 1
1.1 Purpose 1
1.2 Standards & Document Conventions 1
1.3 Glossary 2
2. Technical Stack 4
I. Embedded 4
II. Frontend (mobile app) 4
III. Backend (API + database) 4
3. Front End Design 5
4. Basic Logic 9
5. Data Layer 11
6. Security Specifications 14
7. Performance and Scalability 15
8. Deployment and Infrastructure 15
9. Version Control & Development Workflow 17
10. Technical Specification List 18
11. References 42

Revision History

Name	Date	Reason For Changes	Version
Nolan Tuttle	11/3/25	Initial Document Creation	1.0

Member Name	Email	Team Role
Daniel Grijalva	**Dgrijalva3@my.gcu.edu**	**Software + Cloud Lead**
Nolan Tuttle	**NTuttle1@my.gcu.edu**	**Hardware + Integration Lead**

1. Introduction

1.1 Purpose

PhytoPi is an IoT-based controlled environment system for supporting a plant throughout its entire growth cycle with minimal human intervention. Utilizing embedded hardware and a machine learning system monitoring the plant using a camera, PhytoPi is designed to automatically care for more than 80% of a plant's required maintenance over its lifespan. The goal upon completion of a minimum viable product is to support at least one species of plant throughout its growth cycle from germination to harvest/maturity, or for at least for 70 days.

1.2 Standards & Document Conventions

I. **Requirement Notation**

   a. Functional requirements: Prefixed with FR-XXX.

   b. Non-functional requirements: Prefixed with NFR-XXX.

   c. Technical specifications: Prefixed with TS-XXX.

   d. User stories: Prefixed with USR-XXX.

II. External references use **APA 7th edition** formatting.

III. URLs included with access date when applicable.

1.3 Glossary

API (Application Programming Interface) – A set of endpoints and protocols allowing software components to communicate, such as between the Raspberry Pi and mobile app.

Actuator – A hardware component that performs actions in response to commands, e.g., water pump, fan, or light.

Calibration – The process of adjusting sensors to ensure that their readings match known reference values.

Control Logic – Software rules that automate system actions based on sensor inputs (e.g., turning on a water pump when soil moisture is low).

FR (Functional Requirement) – A requirement specifying a function or behavior that the system must implement.

I²C (Inter-Integrated Circuit) – A serial communication protocol used by microcontrollers and sensors to exchange data.

JWT (JSON Web Token) – A standard for securely transmitting information between parties for authentication and authorization.

Latency – The delay between an event occurring (sensor reading, data transmission) and the system response.

ML (Machine Learning) – Algorithms that analyze plant growth or environmental data to make predictions or decisions.

NFR (Non-Functional Requirement) – A requirement specifying system quality attributes like performance, security, or reliability.

Raspberry Pi 5 (RPi5) – A single-board computer used as the main embedded platform for PhytoPi.

REST (Representational State Transfer) – An architectural style for building networked APIs.

Sensor – A device that measures environmental parameters like temperature, humidity, soil moisture, pH, or light intensity.

Supabase – A cloud service platform providing database, authentication, and real-time features for the system.

Throughput – The amount of data the system can process within a given time frame.

UART (Universal Asynchronous Receiver/Transmitter) – A hardware communication protocol used to transmit data between devices asynchronously.

User Story – A description of a feature or requirement from an end-user perspective, often prefixed with **USR-XXX**.

Validation – The process of ensuring data conforms to expected ranges or formats before being stored or processed.

WebSocket – A protocol enabling real-time, two-way communication between client and server.

Wi-Fi – Wireless networking technology used for communication between the Raspberry Pi and cloud services.

Workflow – The sequence of steps for development, deployment, or data processing.

2. Technical Stack

I. Embedded

a. Platform: Raspberry Pi 5 (64-bit Linux-based OS), ESP32 Microcontroller

b. Programming Languages: C, Bash

c. Libraries/Tools: i2c-dev.h, fcntl.h, unistd.h, ioctl.h, i2cdetect, GNU Debugger

d. Interfaces: I^2C, UART (RS-485), GPIO, ADC (ADS1115)

e. Purpose: Sensor data collection and actuation control

II. Frontend (mobile app)

a. Architecture: REST + WebSocket API Gateway

b. Authentication: OAuth2 / JWT

c. Database: PostgreSQL (hosted via Supabase)

d. File Storage / Realtime Sync: Supabase

III. Backend (API + database)

a. Mobile App: Flutter (Dart)

b. Web Console: Next.js (React-based)

c. Realtime Communication: WebSocket

d. Purpose: Monitoring, control, and user alerts

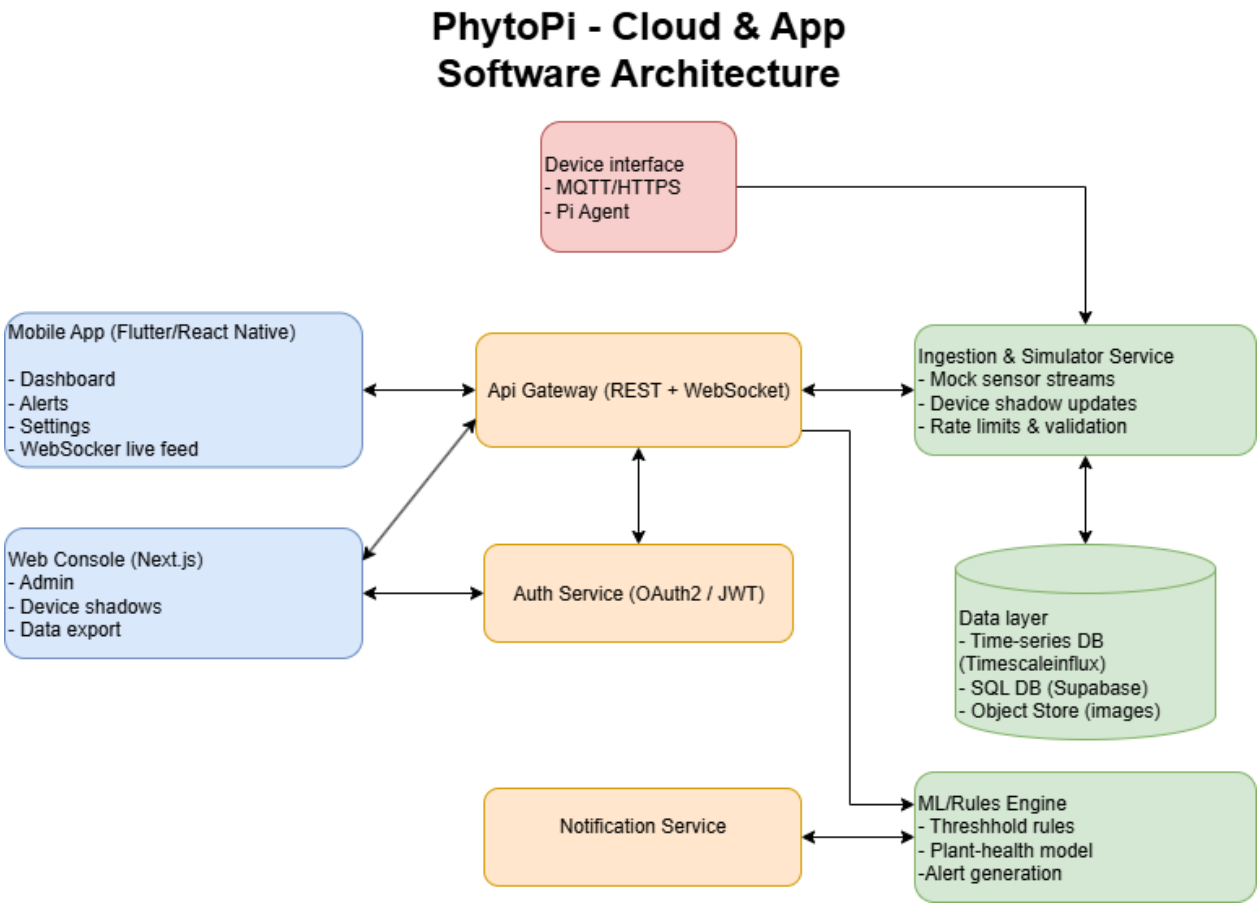

Figure 1: Technical Stack – Software Architecture Diagram

3. Front End Design

I. Overview

a. Mobile Application (Flutter): Used for real-time plant monitoring, manual control actions, and environment alerts.

b. Web Console (Next.js): Designed for administrator-level users to view detailed analytics, logs, and configure the system.

II. UI/UX Design Principles

a. Simplicity: Focus on creating intuitive, simple menu navigation and essential functions such as viewing sensor data and receiving alerts.

b. Consistency: Reuse icons, colors, themes, and layouts between the web and mobile version as well as across menus.
c. Responsiveness: All components should adapt to screen size and interactive elements such as buttons should respond to users immediately.

III. Navigation Structure

a. Dashboard: Displays temperature, humidity, soil moisture, pH, and light levels with color-coded indicators.
b. Control Panel: Manual override for pump, fan, or lighting.
c. Alerts Page: Shows notifications for errors, sensor failures, or critical environmental changes.
d. Settings: User account management, threshold customization, and data history export.

IV. Wireframes

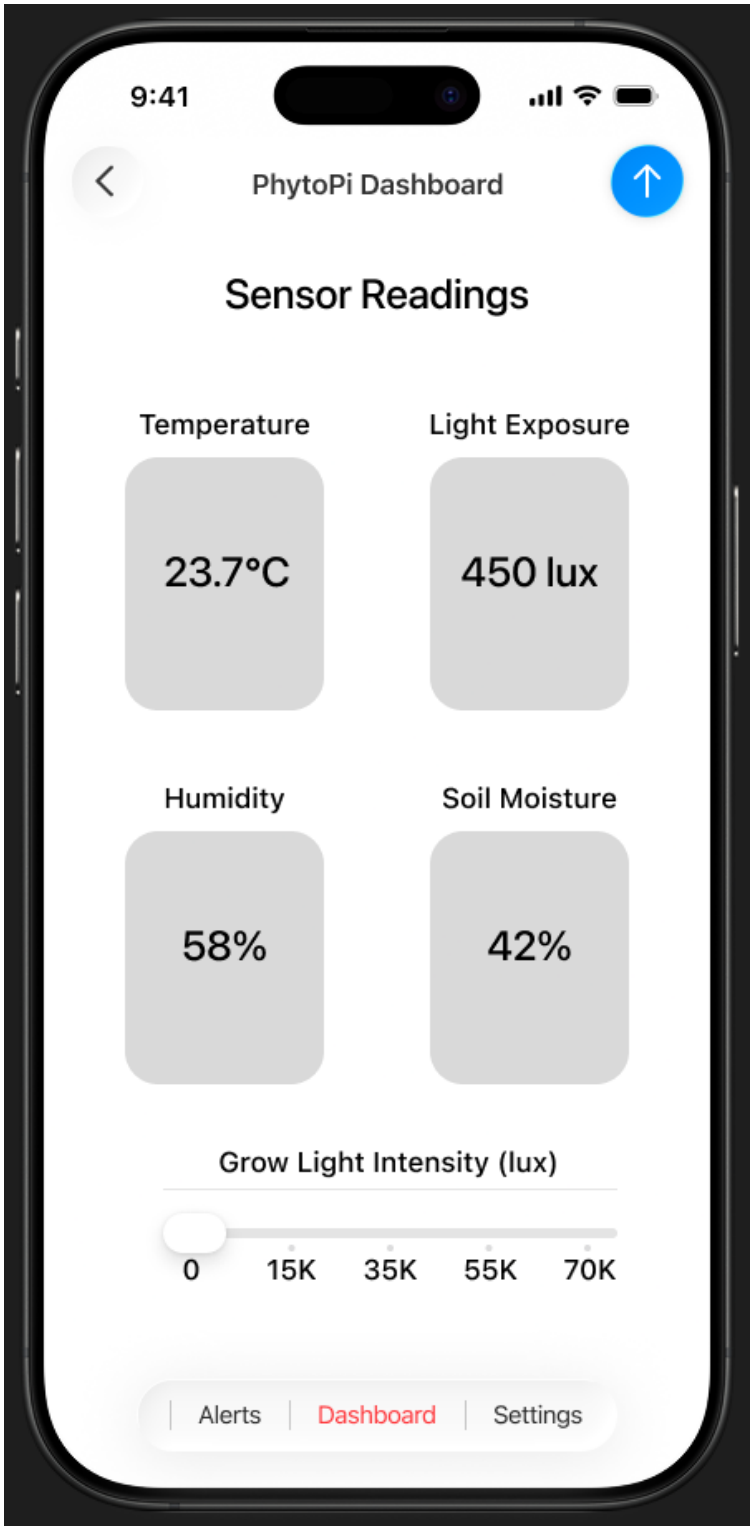

Figure 2: Front End Design – Mobile App Dashboard Wireframe

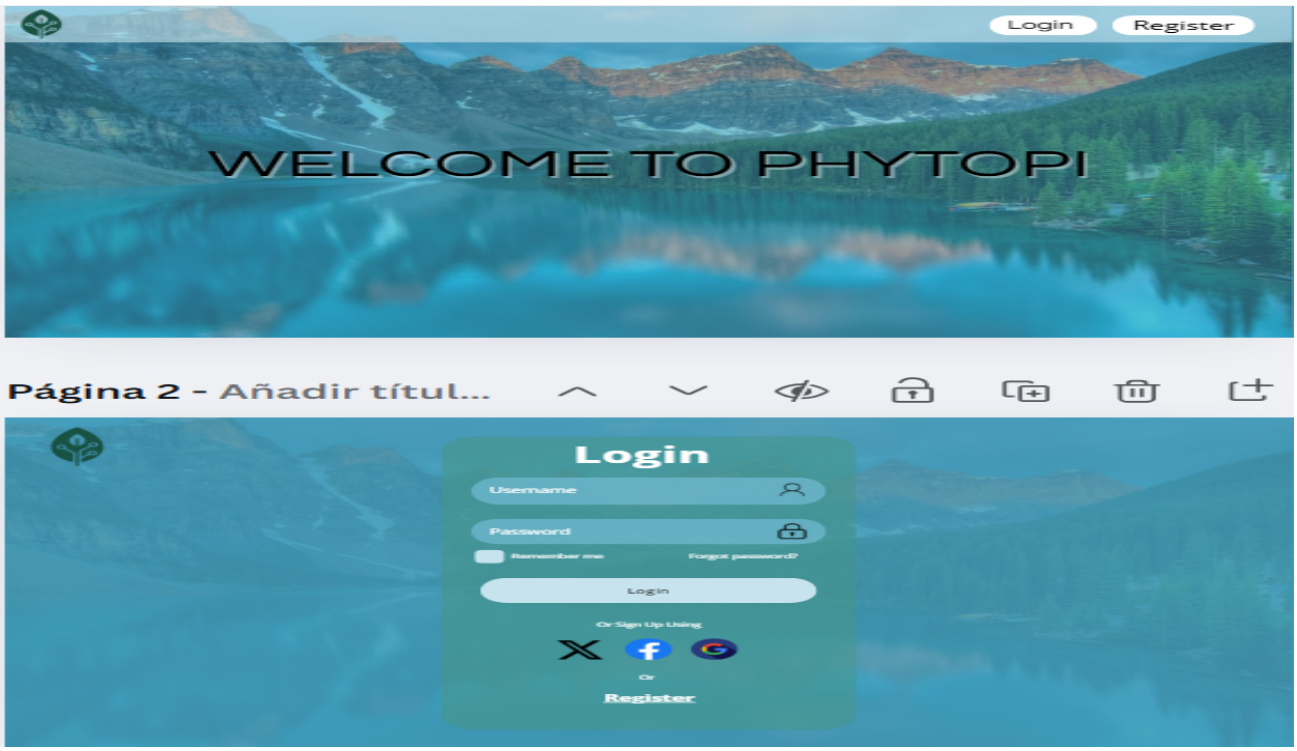

Figure 3: Front End Design – Web Console Login Page Wireframe

Figure 4: Front End Design – Web Console Registration Page Wireframe

4. Basic Logic

I. UML/Flowcharts

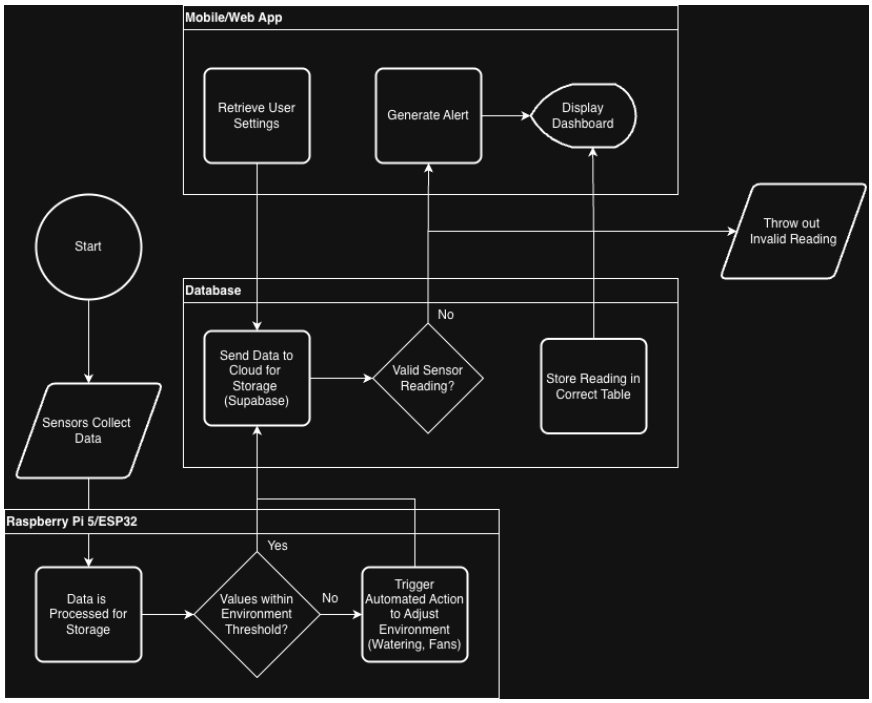

Figure 5: Basic Logic - High-Level Flowchart Logic

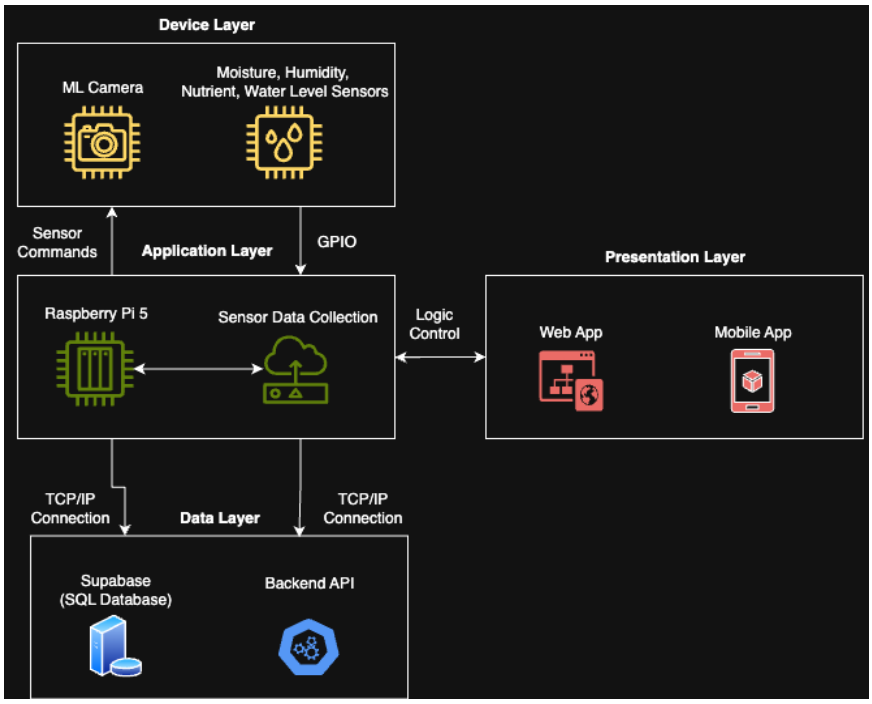

Figure 6: Basic Logic - System Architecture Diagram

II. Error/Exception Handling

a. Sensor Errors

i. Detection: Software exceptions, out-of-range readings

ii. Action: Log error locally on Raspberry Pi

iii. Notification: Send alert to database for mobile app to read

iv. Recovery: Continue normal monitoring state if possible

b. Actuator Errors (Water pump, fan)

i. Detection: Software exceptions, pump/fan fails to respond

ii. Action: Log error locally on Raspberry Pi

iii. Notification: Send alert to database for mobile app to read

iv. Recovery: Pause control logic for actuator until issue is resolved

c. Database Failures

i. Detection: Failed write to database

ii. Action: Retry write automatically, fallback to storing in a local data queue

iii. Notification: Email alert if database remains inaccessible

iv. Recovery: Sync queued data when database becomes available

d. Logging Mechanisms

i. Local storage on Raspberry Pi for hardware/actuator failures

ii. Log database/API transactions in cloud storage

iii. All logs (sensors, actuators, database) are timestamped for debugging purposes

Table 1: API Endpoints

Endpoint	Method	Request Format	Response Format	Description
/sensors/data	POST	{sensor_id, value, timestamp}	{status: “ok”}	Pi sends a new sensor reading
/alerts/new	GET	N/A	{alert_id, type, message, timestamp}	Mobile app fetch for current alerts

/control/pump	POST	{action, timestamp}	{status: “ok”}	Trigger for an environment warning

Authentication for mobile app HTTP requests to the API will be done using JWT tokens, different authorization levels will be available with admins having read/write access to all endpoints and the user-level having access to only read and control actions for their own device.

5. Data Layer

I. Overview

a. The data layer supports persistent storage of sensor readings, actuator states, user preferences, and system logs. It provides the foundation for real-time monitoring, alerting, and control. See *Figure 6* to see the separation of concerns between the data layer and the presentation layer, all data is handled by the application/data access layer and cannot be modified by the user directly, following the N-layer architectural principles.

II. Data Flow

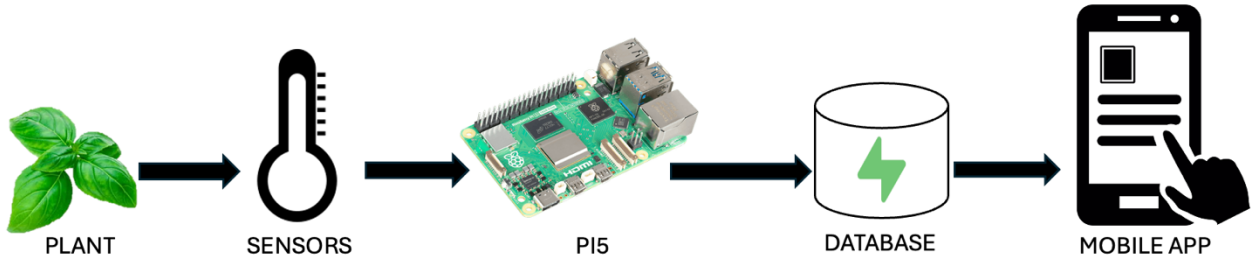

Figure 7: Data Layer – High-Level Data Flow Diagram

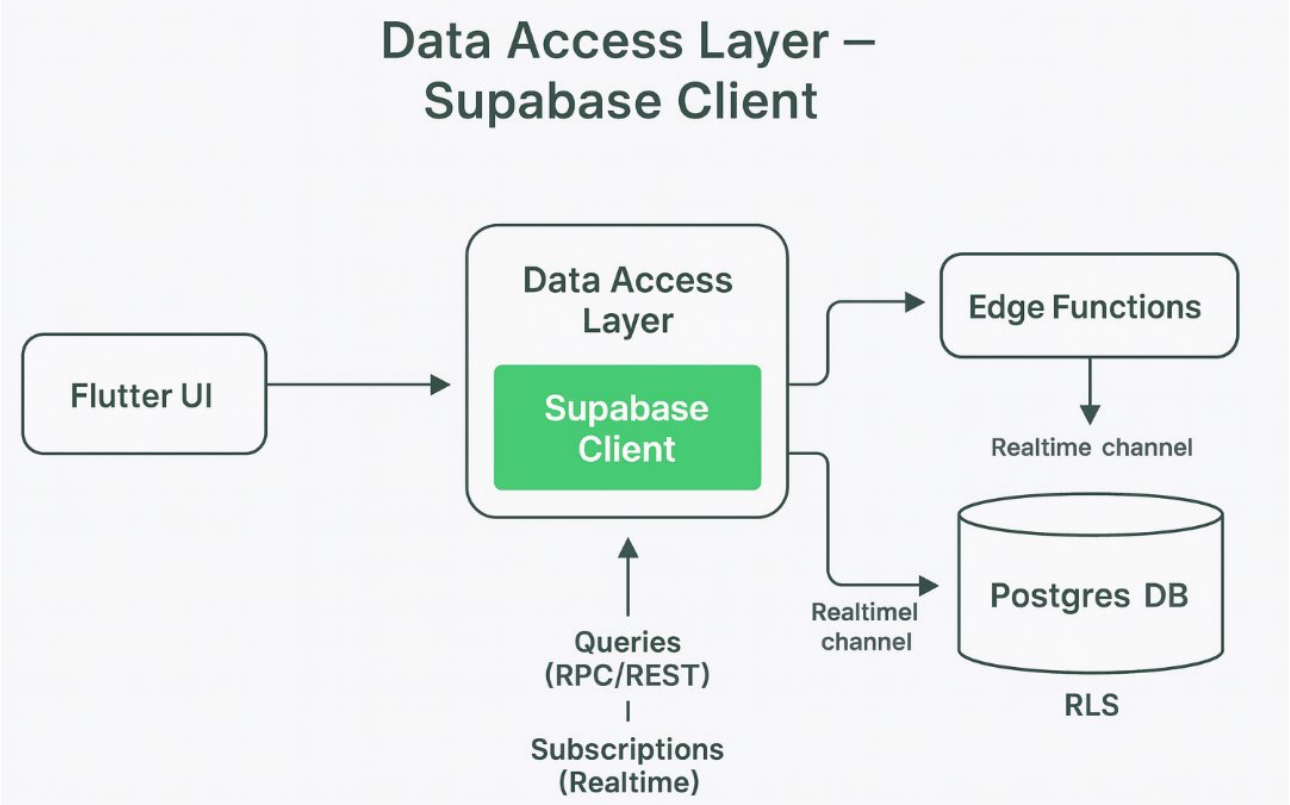

Figure 8: Data Layer – Supabase Client Data Flow Diagram

III. Database Schemas

Table 2: Users Database Schema

Field	Type	Description
id	UUID (Pk)	User ID for authorization
email	varchar(50)	User email for signup
created_on	Date	Registration date

Table 3: Sensors Database Schema

Field	Type	Description
id	UUID (Pk)	Sensor record ID
device_id	UUID (Fk → devices.id)	Source device
timestamp	Date	Timestamp
soil_moisture	Float	Moisture %
temperature	Float	Temperature in Celsius

humidity	Float	Humidity %
light	Integer	Light Lux Value

Table 4: Control Database Schema

Field	Type	Description
id	UUID (Pk)	Control ID
device_id	UUID (Fk → devices.id)	Associated device
light_on	Boolean	Light state
pump_on	Boolean	Pump state
fan_on	Boolean	Fan state

Table 5: Alerts Database Schema

Field	Type	Description
id	UUID (Pk)	Alert ID
device_id	UUID (Fk → devices.id)	Linked device
type	varchar(50)	Alert type (e.g. high temp, low moisture)
message	varchar(50)	Alert description
triggered_at	Date	Timestamp
resolved	Boolean	Whether alert has been acknowledged or not

Table 6: Devices Database Schema

Field	Type	Description
id	UUID (Pk)	Unique device ID
name	varchar(50)	Device name
owner_id	UUID (Fk → users.id)	Owner of the device

IV. Validation Rules

a. temperature must be between 0 C and 60 C
b. humidity between 0% and 100%
c. soil_moisture must be between 0 % and 100 %
d. light_lux maximum at 200,000
e. device_id must exist in devices table
f. Foreign keys have cascaded deletion, meaning that when a device is deleted all foreign key related data is removed.

6. Security Specifications

Authentication: Mobile app users authenticate their sessions by logging in via username and password. On a successful login, a JWT is issued which is used in all future API requests to authenticate that user.

Authorization: The backend verifies the JWT each time a request is received to verify the user is allowed to perform the requested control action.

Storage: All API communications happen over HTTPS and sensitive data, such as user credentials and personal info, is stored encrypted securely in the database.

Compliance: To comply with best security standards, JWT tokens are used for stateless session handling and user passwords are never stored on the app. Open Web Application Security Project (OWASP) mobile app security guidelines are followed when conducting mobile app security tests (OWASP, n.d.).

7. Performance and Scalability

Sensors: System should capture temperature, humidity, soil moisture, and light data at a minimum interval of once per minute.

Data Logic: Automated control actions such as fan control and plant watering must execute within 2 seconds of receiving sensor data.

Mobile App: Refresh and display real-time data about plant health with <2 second latency when updating dashboard.

Database Storage: Sensor data and logs should be stored in the database without data loss, failed writes should be stored in a local queue and automatically retry, syncing the queue with existing data.

Scalability: When scaling for an increased load, Supabase's database can be partitioned to handle multiple plants. The sensor network will have a modular design, allowing more sensors to be added without logic modification by utilizing automatic sensor detection. Dashboards and frequently accessed pages on the mobile/web app can be cached for faster load times, API calls can be limited to throttle request load.

8. Deployment and Infrastructure

V. Environments

a. Development Environment

i. Runs locally on developer's machine with a similar environment to the Pi.

ii. Verifies software changes and new sensor integrations before pushes to staging branch.

iii. Uses a separate local database instance to avoid modifying the production database.

b. Staging Environment

i. Same as the production setup except used for QA/testing.

ii. Deployed on a cloud-hosted VM on Linode that mimics real conditions.

iii. Mobile app connects to staging API endpoints to avoid modification of production database, used for validation before production release.

c. Production Environment

i. Live environment where the system interacts with a plant.

ii. Raspberry Pi 5 in physical environment continuously runs the control software.

iii. Connects to the production database, using it for data storage and authentication.

VI. Deployment

a. Business Logic

i. Source code managed using Git.

ii. Developers will push updates to main branch, triggering automated deployment pipeline through Github Actions.

b. Mobile/Web App

i. Flutter mobile app built and released through Android Studio or Xcode to test device compatibility and mobile app functionality.

ii. Next.js web console hosted via Linode cloud virtual machine.

VII. Infrastructure Requirements

a. Hardware

i. Raspberry Pi 5, sensors, actuators, power supply, network connectivity.

b. Cloud Services

i. Supabase (PostgreSQL, authentication, file storage, WebSocket API), Linode VM.

c. Network

i. Stable Wi-Fi connection for cloud communication.

9. Version Control & Development Workflow

I. Version Control

a. Git Branch Structure

- **i.** Main branch contains tested, deployed, and production-ready code
- **ii.** Staging branch contains completed features from development branches before production deployment for testing
- **iii.** Feature branches contain individual features/fixes with descriptive names

II. Development Workflow

a. Developers code in isolated feature branches for individual features or fixes, with extensive comments and readability-focused style conventions

b. Each feature branch is unit tested locally on the development environment. Automated tests are run on the staging branch via GitHub Actions before merging

c. Pull requests must be submitted for review before merging into main. Reviews verify code quality, functionality, and adherence to project requirements.

d. Merges into main trigger the automated deployment pipeline through GitHub Actions. The pipeline handles building the mobile app, deploying backend code to the Linode VM, and syncing database changes as needed.

e. All changes, features, and bug fixes are documented in commit messages and the project's Discord to maintain clarity across the team.

10. Technical Specification List

Technical Specification ID	Technical Specification	Functional Requirement	Functional/Non-functional Requirement ID	Related User Story	Related User Story ID
TS-001	Create complete electrical wiring diagram for all hardware components.	The system shall provide a complete electrical wiring diagram for all hardware components.	FR-001	As a developer I need to design an electrical wiring diagram so that I can ensure safety, organization, and proper function of hardware components.	USR-001
TS-002	Highlight signal and power connections in wiring diagram for clarity.	The system shall highlight all important connections such as signal and power so that the diagram is properly organized.	FR-002		

TS-003	Add safety annotations for high-voltage or sensitive components.	The system shall indicate safety measures for components prone to damage or with high voltage.	FR-003		
TS-004	Ensure diagram output is legible and printable in A4 format.	Output must be legible and printable in A4 format.	FR-004		
TS-005	Generate to-scale 3D model of all hardware components and layout.	The system must generate a to-scale model representing all hardware components and their arrangement.	FR-005	As a developer I need to design a to-scale model of the system so that I can know which materials will be needed and the dimensions of the system.	USR-002

TS-006	Export model measurements in standard engineering units for material reference.	The model must export measurements and dimensions in standard engineering units for reference in material selection.	FR-006		
TS-007	Limit model file size to 100 MB for sharing purposes.	Model file size must not exceed 100 MB for sharing purposes.	FR-007		
TS-008	Create database schema for pH readings including sensor ID and pH value.	The schema must include attributes for sensor ID, and pH values.	FR-008	As a system I need to create a database schema for soil nutrient readings so that the data is easily queryable.	USR-003
TS-009	Create database schema for soil moisture readings including sensor ID and moisture percentage.	The schema must include attributes for sensor ID, and moisture percentages.	FR-009	As a system I need to create a database schema for soil moisture readings so that the data is	USR-004

				easily queryable.	
TS-010	Implement main navigation menu for accessing all system menus.	The system shall allow users to navigate between all menus using the main navigation menu.	FR-010	As a user, I want dashboards and charts to be responsive and intuitive to navigate so that I can access information quickly.	USR-005
TS-011	Ensure page navigation occurs in under 1 second on stable internet connection.	Navigating between pages shall occur in under 1 second on a stable internet connection.	NFR-001		

TS-012	Provide toggle options for enabling/disabling alert notifications.	The system shall allow users to enable and disable notifications for alerts from the system.	FR-011	As a user, I want to configure alert preferences so that I only get notifications that matter to me.	USR-006
TS-013	Persist user notification preferences across sessions in database.	The system shall store and apply user's notification preferences across sessions.	FR-012		
TS-014	Maintain notification settings error rate below 1% during usability tests.	The notification settings shall have an error rate below 1% in usability testing.	NFR-002		
TS-015	Generate graphical growth trend charts from plant data.	The system shall generate and display graphical growth trends using plant data	FR-013	As a user, I want to see historical growth data and trend analysis so that I can make informed care decisions.	USR-007

TS-016	Render growth trend graphs in under 5 seconds after selecting time period.	Graphs shall render in under 5 seconds after time period selection	NFR-003		
TS-017	Display color-coded alerts for plant or environment status.	The system shall display color-coded alerts to indicate plant or environment status.	FR-014	As a user, I want color-coded indicators of plant stress so that I can quickly see which plants need attention.	USR-008
TS-018	Update color-coded alerts within 500 milliseconds of status change.	Color changes shall occur within 500 milliseconds of alert status update.	NFR-004		
TS-019	Trigger alerts when environmental readings exceed predefined thresholds.	The system shall generate alerts when environmental readings exceed predefined thresholds.	FR-015	As a user, I want push notifications to alert me of unexpected environment conditions so that I can act	USR-009

				promptly even when away.	
TS-020	Deliver alerts to user devices within 2 seconds of detection.	Alerts shall be generated and delivered within 2 seconds of data detection.	NFR-005		
TS-021	Push notifications to user devices when thresholds are exceeded.	The system shall push notifications to the user's device reading thresholds are exceeded	FR-016		

TS-022	Display real-time sensor data on the main dashboard.	The system shall display real-time data on the main dashboard.	FR-017	As a user, I want a dashboard showing moisture, temperature, light, and nutrient levels so that I can check my plants remotely.	USR-010
TS-023	Refresh dashboard data with less than 1 second delay from sensor input.	Dashboard data refresh rate shall not exceed 1 second delay from actual sensor input.	NFR-006		
TS-026	Store machine learning predictions in cloud database.	The system shall store all machine learning predictions in the cloud database.	FR-018	As a system, I need to store ML predictions in the cloud database so that it can be accessed to display alerts or dashboard information.	USR-011

TS-027	Ensure storing ML predictions takes no longer than 500 milliseconds.	Storing machine learning predictions in the cloud should not take longer than 500 ms.	NFR-007		
TS-028	Validate incoming sensor data against defined acceptable ranges before storage.	The system must check each section of incoming sensor data against defined acceptable ranges before storage.	FR-019	As a system, I need to validate incoming data so that corrupt or out-of-range data is flagged or rejected.	USR-012
TS-029	Flag or reject missing or out-of-range sensor data points.	The system must flag or reject data points that are missing, or outside threshold values.	FR-020		
TS-030	Complete data validation process in under 100 milliseconds per reading.	Data validation process must complete in under 100ms per reading.	NFR-008		

TS-031	Maintain data validation accuracy of at least 99%.	Data validation accuracy >= 99%	NFR-009		
TS-032	Provide secure API endpoints for retrieving sensor data, alerts, and ML predictions.	The system shall provide secure API endpoints for retrieving sensor data, alerts, or ML predictions.	FR-021	As a system, I need to provide secure endpoints to retrieve sensor and ML data so that the business logic layer can access information safely.	USR-013
TS-033	Require authentication tokens for all API data access requests.	The system shall require authentication tokens for each data access requests.	FR-022		
TS-034	Ensure API response times do not exceed 1 second for 95% of requests.	API response times shall not exceed 1 second for 95% of requests.	NFR-010		

TS-035	Enforce HTTPS and encrypt all data in transit for all endpoints.	All endpoints should use HTTPS and enforce encryption of data during transit.	FR-023		
TS-036	Store automated or manual control actions with timestamp, type, and description.	The system must store each automated or manual control action along with its timestamp, type, and description.	FR-024	As a system, I need to store timestamps and logs of all control actions so that the system trends can be analyzed._	USR-014
TS-037	Organize control action data in structured format for analytics and UI usage.	The system must organize control action data in a structured format for analytics or use in UI.	FR-025		
TS-038	Ensure storing control logic actions occurs in under 500 milliseconds.	Storing a received control logic action must occur in <= 500ms	NFR-011		

TS-039	Retain stored control logic actions in database for at least 90 days.	Stored control logic actions must be retained in the database for >= 90 days	NFR-012		
TS-040	Record all incoming sensor data with timestamps and sensor type.	The system must record all incoming sensor data along with their timestamps and sensor type.	FR-026	As a system, I need to store all sensor readings in a cloud database so that historical and real-time data are accessible to other systems._	USR-015
TS-041	Maintain database writing latency under 200 milliseconds.	Database writing latency must not exceed 200ms	NFR-013		

TS-042	Record all incoming sensor data with timestamps and sensor type.	The system must record all incoming sensor data along with their timestamps and sensor type.	FR-027	As a system, I need to feed camera plant images into the ML model so that growth and stress can be detected automatically. _	USR-016
TS-043	Ensure database writing latency does not exceed 200 milliseconds.	Database writing latency must not exceed 200ms	NFR-014		
TS-044	Define minimum and maximum thresholds for each sensor type to ensure valid readings.	The system shall define minimum and maximum thresholds for each sensor type (e.g., soil moisture, pH) to ensure valid readings.	FR-029	As a system, I need to apply thresholds and rules to sensor readings so that automated interventions are accurate in the event of sensor failure or unexpected conditions._	USR-017

TS-045	Validate sensor thresholds in under 100 milliseconds per reading.	Sensor threshold validation shall occur in under 100 ms per reading.	NFR-015		
TS-046	Record every automated action executed by control logic.	The system shall record every automated action executed by control logic.	FR-030	As a system, I need to log all automated actions taken by control logic so that historical trends are available for analysis._	USR-018
TS-047	Timestamp each control action and include outcome in log entry.	The system shall timestamp and include outcome for each log entry.	FR-031		
TS-048	Store action logs with capacity to retain at least 30 days of data.	The storage capacity for action logs should store >= 30 days	NFR-016		

TS-049	Retrieve logs of control logic actions within 2 seconds.	Retrieving logs of control logic actions should take <= 2 seconds.	NFR-017		
TS-050	Continuously compare sensor readings against expected ranges.	The system shall continuously compare sensor readings against expected readings.	FR-032	As a system, I need to detect when readings from sensors are not within expected range so that notifications can be sent promptly.	USR-019
TS-051	Flag abnormal readings and generate corresponding alert messages.	The system shall flag abnormal readings and generate alert messages.	FR-033		
TS-052	Log anomalies in storage for later review.	The system shall log anomalies in storage for later review.	FR-034		

TS-053	Detect anomalies with latency under 2 seconds.	The latency for anomaly detection is <= 2 seconds.	NFR-018		
TS-054	Maintain false positive rate of anomaly detection below 5%.	The false positive rate for anomaly detection is <= 5%.	NFR-019		
TS-055	Retain stored anomaly detection alerts for at least 30 days.	Stored anomaly detection alerts are retained >= 30 days.	NFR-020		
TS-056	Compare soil moisture readings against defined thresholds.	The system shall compare soil moisture readings against defined thresholds.	FR-035	As a system, I need to evaluate soil moisture data so that the system can trigger watering alerts or automation.	USR-020
TS-057	Trigger automatic watering when soil moisture is below threshold.	The system shall trigger automatic watering when moisture is below threshold.	FR-036		

TS-058	Evaluate soil moisture levels against thresholds in under 5 seconds.	The system shall evaluate soil moisture levels against threshold in < 5 seconds.	NFR-021		
TS-059	Ensure automatic watering system reliability is at least 99%.	The automatic watering system reliability should be >= 99%	NFR-022		
TS-060	Automatic watering system responds within 15 seconds of unexpected moisture levels.	The automatic watering system should respond within 15 seconds of unexpected soil moisture levels.	NFR-023		

TS-061	Transmit validated sensor readings to Raspberry Pi every 10 seconds using secure protocol.	The system shall transmit validated sensor readings to the Raspberry Pi every 10 seconds using a secure communication protocol	NFR-024	As a system, I need to transport sensor readings to the Raspberry Pi so that downstream logic systems have accurate data for use._	USR-021
TS-062	Maintain minimum 99.5% successful transmission rate under stable network conditions.	The system shall maintain a minimum 99.5% successful transmission rate under stable network conditions.	NFR-025		
TS-063	Ensure data transmission latency does not exceed 300 milliseconds per packet.	Data transmission latency shall not exceed 300 ms per packet.	NFR-026		

TS-064	Provide calibration routine for each connected sensor.	The system shall provide a calibration routine for each sensor connected.	FR-037	As a system, I need to calibrate all sensors so that readings are consistent and trustworthy._	USR-022
TS-065	Verify calibration accuracy against known reference values.	The system shall verify calibration accuracy against known reference values.	FR-038		
TS-066	Complete calibration routine within 5 seconds per sensor.	Calibration routine shall complete within 5 seconds per sensor.	NFR-027		
TS-067	Maintain system uptime above 99% during calibration.	The system uptime during calibration > 99%	NFR-028		

TS-068	Read temperature from sensors at specified intervals.	The system shall read temperature from temperature sensors at specified intervals.	FR-039	As a system, I need to collect temperature and humidity readings so that environmental monitoring is complete._	USR-023
TS-069	Read humidity from sensors at specified intervals.	The system shall read humidity from humidity sensors at specified intervals.	FR-040		
TS-070	Store and timestamp temperature and humidity readings in local storage.	The system shall store and timestamp temperature and humidity readings in local storage for later use.	FR-041		
TS-071	Maintain sensor accuracy of ±1.0°C for temperature readings.	Sensor accuracy must be ±1.0°C for temperature readings.	NFR-029		

TS-072	Maintain sensor accuracy of ±5% for humidity readings.	Sensor accuracy must be ±5% for humidity readings.	NFR-030		
TS-073	Ensure system uptime above 99% when collecting readings.	System uptime when collecting readings is >99%	NFR-031		
TS-074	Read soil moisture values from moisture sensors at specified intervals.	The system shall read soil moisture values from moisture sensors at specified intervals.	FR-042	As a system, I need to collect soil moisture readings so that control logic and ML have accurate input._	USR-024
TS-075	Store and timestamp soil moisture readings in local storage.	The system shall store and timestamp soil moisture readings in local storage for later use	FR-043		
TS-076	Ensure soil moisture sensor readings have latency under 1 second.	The sensor readings have a latency of < 1 second.	NFR-032		

TS-077	Maintain uptime above 99% for soil moisture monitoring.	The uptime for soil moisture monitoring is > 99%	NFR-033		
TS-078	Analyze historical pH data to detect imbalances.	The system shall analyze historical pH data to determine imbalances.	FR-044	As a system I need to analyze historical soil nutrient and pH data so that I can predict when a plant will require replenishment.	USR-025
TS-079	Generate recommendations for pH adjustment based on analysis.	The system shall generate recommendations for adjustment based on analysis results.	FR-045		
TS-080	Achieve analysis accuracy of at least 95% against known nutrient tests.	Analysis accuracy shall achieve at least 95% when validated against known nutrient test results.	NFR-034		
TS-081	Complete analysis	Analysis computations	NFR-035		

	computations within 3 seconds per plant dataset.	shall complete within 3 seconds per plant dataset.			
TS-082	Compare optimal soil pH readings against detected pH levels.	The system shall compare optimal pH soil readings against detected pH levels.	FR-046	As a system I need to evaluate pH readings so that I can determine when a plant requires additional nutrients.	USR-026
TS-083	Record nutrient-related alerts and anomalies for trend analysis.	The system shall record nutrient-related alerts/anomalies for trend analysis	FR-037		
TS-084	Maintain pH level detection accuracy of at least 90%.	The accuracy of pH level detection >= 90%	NFR-036		
TS-085	Retain alert logs for at least 30 days.	The alert log must be retained for >= 30 days.	NFR-037		

TS-086	Collect soil nutrient readings at specified intervals.	The system shall collect soil nutrient readings at specified intervals.	FR-038	As a system, I need to collect soil nutrient and pH readings so that reliable data is available for nutrient alerts and ML._	USR-027
TS-087	Collect soil pH readings at specified intervals.	The system shall collect soil pH readings at specified intervals.	FR-049		
TS-088	Store and timestamp soil nutrient and pH readings in local storage.	The system shall store and timestamp soil nutrient and pH readings in local storage for later use.	NFR-038		
TS-089	Maintain soil nutrient sensor accuracy of ±5%.	The soil nutrient sensor has an accuracy of ±5%	NFR-039		
TS-090	Maintain pH sensor accuracy of ±0.2.	The pH sensor has an accuracy of ±0.2.	NFR-040		
TS-091	Ensure data integrity of sensor readings	The data integrity of sensor reading is ensured so that	NFR-041		

	with no data loss over 24 hours.	no data loss occurs over a 24 hour period			

Jira Link: https://plantgrowthmanager.atlassian.net/jira/software/projects/D4N/summary

11. References

Open Web Application Security Project. (n.d.). *OWASP Mobile Application Security (MAS) project*. OWASP. Retrieved November 2, 2025, from https://owasp.org/www-project-mobile-app-security/